密级

秘密

版本记录

日期	版本	修改人	操作说明
20250425	1.0	李宗	初稿

引言

业务背景

功能需求

按照公司架构办日志规范提取需求点。

1. 提供logproxy接口满足不同类型的日志记录，支持不同类型错误记录在不同的日志
2. 错误号重新定义
3. remoter、front、bizproc接口调用改造计划，需匹配正确的接口和错误号
4. remoter不得出现运维错误，需要将remoter回调给上层，其他模块可以出现程序错误、运维错误
5. 支持不同类型日志路径可配置
6. 按照最新的格式产生日志
7. 支持日志归档和不同类型日志清理天数不同的配置？？？

非功能需求

1. 性能准出标准

   待补充

2. 安全性需求

敏感信息的屏蔽要做到不可逆性

详细设计

日志规范化

日志使用者（第一维度）	日志类别	日志级别	用途	日志场景（第二维度）	说明
业务人员	业务信息日志	INFO/ERROR	定位业务异常、监控批处理进度	请求执行报错，联机异常不需要运维干预；批处理打点	用户登录密码错误、用户无权限等； 批处理执行过程中的打点日志，用于检测批处理的执行状态和性能
业务人员	操作日志	INFO/ERROR	合规稽核与操作追溯	运维操作人员操作运维平台、业务操作人员操作业务系统	用于系统合规稽核
运维人员	系统运维日志	ERROR/FATAL	排查硬件/网络/配置故障	请求执行报错，联机异常需要运维干预	操作系统、磁盘、内存、网络等基础设施和硬件错误； 运维参数设置错误
运维人员	事件日志	EVENT	记录系统启停及关键状态变更	启动、停止、状态变更、运维操作对象（回退、限流）	记录重要事件，用于问题排查
运维人员	警告日志	WARN	提前发现潜在风险并处理	证书快到期、队列排队、超时重试	系统自动检测产生的日志，需要运维人员处理的风险
运维人员	链路日志	INFO	性能分析与跨系统故障追踪	基于可观测性做的后台埋点日志，主要作用是性能分析、故障排查和依赖分析	链路日志、DB慢请求链路、分布式缓存慢请求链路
开发人员	程序运行日志	INFO/ERROR/FATAL	定位代码缺陷或组件异常	包括请求执行中的程序缺陷错误和非服务技术组件（比如SDK）日志。前者用于开发人员定位和解决错误，后者主要用于排查异常。	后台服务参数输入超限、SQL语句执行错误、数据异常错误。非服务技术组件主要是一些技术组件公共包，被其它系统调用，写日志是为了分辨是哪里的异常。
开发人员	调试日志	DEBUG	开发阶段功能调试与问题复现	开发调试	开发人员用于调试系统增加的日志，通常只在开发期打印

编号	日志级别	用途
1	DEBUG	用于记录详细的调试信息，通常只在开发阶段使用。
2	INFO	用于记录一般性的信息，一般不需要立即人工干预，用于后续统计、分析、稽核。
3	EVENT	用于记录系统重要事件，例如系统启动、系统停止、配置变更、重要状态变更等。
4	WARN	用于记录警告信息，表示系统运行中可能出现问题和风险，但不会导致程序崩溃。
5	ERROR	用于记录错误信息，表示程序运行中出现严重问题，可能导致部分功能无法正常工作。
6	FATAL	用于记录致命错误信息，表示程序运行中出现极其严重的问题，系统可能无法正常工作。

日志分为两部分：

日志头部信息：

日志头部信息（log_header，日志中只有字段值，没有字段的名称，字段值之间采用空格分隔）：			
字段英文名称	字段中文名称	字段描述	字段格式
log_version	日志版本信息	日志数据的版本信息，用于日志格式识别和兼容；本日志规范初始版本V2.0	V+版本号（比如初始版本为：V2.0）
timestamp	**日志时间戳（时间信息）**	**日志时间戳信息**	**yyyy-MM-dd HH:mm:ss.SSSSSS（3位为毫秒，6位为微秒；如果产品不需要到微秒，可以省略微秒）**
log_level	日志级别	日志级别，选项：DEBUG|INFO|EVENT|WARN|ERROR|FATAL	采用选项之一，不允许为空
server_info	服务信息（运行期空间信息）	进程的运行期信息，包括：服务器IP地址、端口、进程名称、线程编号、分片编号、节点编号等；	IP：端口-进程名称-T线程编号-P分片编号-N节点编号；如果没有分片编号，可以置0或者空；如果无节点编号，也可以置0或者空
location	日志发生位置（开发期空间信息）	程序对应的开发期信息，标识错误产生的源头信息。 包括：系统（或者子系统）名称（编号）、接口（功能）名称、方法（函数）名称、代码行； 第一级"系统名称"是接口（功能）之上的最小模块单元。	系统（或者子系统）名称（编号)-接口名称（功能名称）-M方法名称-L代码行； 具体名称的样式由每个系统自己定（有些有英文名称，有些是数字编号）；发生问题的方法名称和代码行如果不能获取到，可以填空
format	日志信息格式	**空格分隔或JSON格式（LDP为空格分隔，其它为JSON格式）：** **1）当为JSON格式时，日志信息分为两个Key（info和ext），不需要外面套一层log_info。info信息以数组方式表示（不需要Key值），ext以键值对的方式表示（原因是ext是可扩展的，info信息不可扩展和删减）；** **2）当为空格格式时，info和ext这两个Key以及内部的字段名（Key）都省略，全部只有值，值与值之间用空格（1个空格）隔开（ext字段信息需要每个产品维护好）。空格分隔时，如果数据带有空格，数据整体用[](中括号）包裹。**	JSON格式为：FJ，空格格式为：FS
type	日志类型	日志类型，决定具体的日志信息格式 0 - 调试日志（DEBUG） 1 - 链路日志 （INFO） 2 - 业务信息日志 （INFO） 3 - 事件日志 （EVENT）（包括关键事件、启动） 5 - 警告日志 （WARN、ERROR） 7 - 系统运维日志（ERROR） 9 - 程序运行日志 （ERROR、FATAL）	**格式：TP+类型（比如调试日志为TP0，链路为TP1）**

日志信息

分为两部分info ext

字段英文名称	字段中文名称	字段描述
通用信息(info)	日志固定信息	按照日志类型定义
扩展信息(ext)	**日志辅助信息；用于日志扩展，每个产品可以在扩展信息中增加字段。**	按照日志类型定义

日志统一格式：

```
log_version timestamp log_level processId-Ttid systemName-moduleName-MfunName-LlineNum formatType type info ext
```

平台调试日志(.debuglog)

开发调试

调试日志信息(type=0)：		
通用信息(info)		
~~**timestamp**~~	~~**日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS**~~	**必填**
version	**当前系统版本编号**	必填
charset	**扩展信息中出入参的中文字符编码** **0为GB18030，1为utf-8，不填为utf-8**	**选填，没有填"-"（减号，表示utf-8）**
扩展信息(ext)		
ext	调试信息	选填，没有填""

日志格式

```
log_version timestamp log_level processID-Ttid systemName-moduleName-MfunName-LlineNum formatType TP0 version 1 ext
```

```
V2.0 2025-09-12 21:17:03.138490 DEBUG 8960-T7589 MOnMessage-Lfront/utils.cpp:116 FS TP0 V3.0 1 131 [调试 (value=1)]
```

ext：调试号 调试信息

链路日志(.tracelog)

基于可观测性做的后台埋点日志，主要作用是性能分析、故障排查和依赖分析

链路日志信息(type=1)：		
通用信息(info)		
trace_id	区分调用链消息的唯一标识，只能是字母和数字的组合； 一个请求的唯一标识，一个调用链中只有一个唯一的TraceID，所有TraceID相同的消息构成一个调用链	必填
span_id	跨度层级； 一个请求从发出到应答回来称为一个跨度（span），spanID用于区分不同的span，从0开始，每增加一个跨度，spanID就加一层，如0,0.1,0.1.1...，如果在一次消息处理中有多个请求被发出，则SpanID递增规则为：0.1,0.2,0.3...。 对于主动发起请求的服务填-1	必填
depot_id	跨度里的层级；在对接BAR情况下，必须要填正确。在合并BAR和AR的场景下，用来标识请求和应答的depot_id，比如r0、a4等。隐含报文类型，r：请求，a：应答，即CRES的T2/F2插件的SR和CS可以合成请求，CR和SS可以合成应答	选填，没有填"-"（减号）
module_type	CS、SR、SS、CR； SR（Service Receive）：作为服务端接收到请求的时刻 SS（Service Send）：作为服务端完成并发送响应的时刻。 CS（Client Send）：作为客户端发送请求的时刻 CR（Client Receive）：作为客户端接收到服务端响应的时刻。	选填，没有填"-"（减号）
~~timestamp~~	~~日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS~~	~~必填~~
response_status	0表示成功，1表示失败	必填
version	当前系统版本编号	必填
server_name	服务名称（服务调用信息）： T2: function_id#sub_system_no#system_no#branch_no T3: function_id#G#S#V#P HTTP: url	必填
app_type	0.unkonw 1.iar 2.jres-mvc 3.jres-svr 4.ar 5.bar 6.ls 7.as 8.db 9.ldp	必填
~~operator_id~~	~~操作员编号~~	~~选填，没有填"-"（减号）~~
~~cust_id~~	~~客户编号~~	~~选填，没有填"-"（减号）~~
charset	扩展信息中出入参的中文字符编码 0为GB18030，1为utf-8，不填为utf-8	选填，没有填"-"（减号，表示utf-8）
response_time	响应时间，微秒	必填
clean_flag	清理标志；0：不清理，1：只清理应答体，2：只清理请求体，3：都清理，包括调用链； 主要是为了节省空间（长度过长，或者内存交易节省空间）	必填
扩展信息(ext)		
error_no	错误编号	必填，正常返回为0
error_info	错误信息（包括相关参数）	选填，没有填写""(或者不显示该字段)
error_infox	记录错误扩展信息，一般为底层报错信息（错误号+错误信息）等	选填，没有填写""(或者不显示该字段)
app_protocol	业务包类型：http/T2/T3/ldp	选填，clean_flag为3时，填空(或者不显示该字段)
req	请求包BASE64	选填，clean_flag为1、3时，填空(或者不显示该字段)
rep	应答包BASE64	选填，clean_flag为2、3时，填空(或者不显示该字段)
refs	refs：调用链信息数组；数组每一项表示一次调用的信息 app_protocol(业务包类型：http/T2/T3/ldp), req（请求包BASE64）,rep（应答包BASE64）, response_time（响应时间，微秒,-1表示超时），error_no（错误编号), error_info（错误信息), span_id, span_type（调用类型）, charset	选填，两种情况为空(或者不显示该字段），一种是clean_flag为3时，另外一种是没有子链路调用时

日志格式

```
log_version timestamp log_level processID-Ttid systemName-moduleName-MfunName-LlineNum formatType TP1 trace_id spanid deport_id module_type responseStatus version serverName appType charset error_no error_info error_infox app_protocol req_body rep_body refs
```

```
V2.0 2025-09-12 21:17:03.138490 INFO 48860-T2789 - FS TP1 0xf12938580607 0.1 0 1 V3.0 11 ldp_bizproc 9 1 0 1542 [转发失败] - ldp reqbase64nidjfllg94885jdfn - [.TName bizproc_worker-1 .Type Req .Eime 1719370 .MergeRsp 1 .CommPattern 0x21 PluginRecv 1757986726860887070 BizRspStart 16530 BizRspEnd 30 ForwardRsp 130 PluginSend 550]
```

由于新格式在refs外已经包含了部分信息，所以，将timelog中的线程id、错误码删除。

如果是业务返回错误，错误信息暂时为空，这里模拟的是平台错误。链路信息行号信息为空。

业务日志(.bizlog)

请求执行报错，联机异常不需要运维干预；批处理打点

通用信息(info)		
trace_id	区分调用链消息的唯一标识，只能是字母和数字的组合； 一个请求的唯一标识，一个调用链中只有一个唯一的TraceID，所有TraceID相同的消息构成一个调用链	必填
span_id	跨度层级； 一个请求从发出到应答回来称为一个跨度（span），spanID用于区分不同的span，从0开始，每增加一个跨度，spanID就加一层，如0,0.1,0.1.1...，如果在一次消息处理中有多个请求被发出，则SpanID递增规则为：0.1,0.2,0.3...。 对于主动发起请求的服务填-1	必填
depot_id	跨度里的层级；在对接BAR情况下，必须要填正确。在合并BAR和AR的场景下，用来标识请求和应答的depot_id，比如r0、a4等。隐含报文类型，r：请求，a：应答，即CRES的T2/F2插件的SR和CS可以合成请求，CR和SS可以合成应答	选填，没有填"-"（减号）
~~timestamp~~	~~日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS~~	~~必填~~
response_status	0表示成功，1表示失败	必填
version	当前系统版本编号	必填
server_name	服务名称（服务调用信息）： T2: function_id#sub_system_no#system_no#branch_no T3: function_id#G#S#V#P HTTP: url	必填
operator_id	操作员编号	选填，没有填"-"（减号）
cust_id	客户编号	选填，没有填"-"（减号）
charset	扩展信息中出入参的中文字符编码 0为GB18030，1为utf-8，不填为utf-8	选填，没有填"-"（减号，表示utf-8）
扩展信息(ext)		
error_no	错误编号	选填，没有填写""(或者不显示该字段）；没有错误编号的场景主要是批处理的信息日志打印
error_info	错误信息（包括相关参数）	选填，没有填写""(或者不显示该字段）；同上
error_infox	记录错误扩展信息，一般为底层报错信息（错误号+错误信息）等	选填，没有填写""(或者不显示该字段）

日志格式

```
log_version timestamp log_level processID-Ttid systemName-moduleName-MfunName-LlineNum formatType TP2 trace_id spanid deport_id responseStatus version server_name operator_id cust_id charset error_no error_info error_infox
```

```
V2.0 2025-09-12 21:17:03.138490 ERROR 4657-T2950 MF973001-Ls_uftmdb_uses_lsu_sespolling_flow.cpp,455 FS TP2 0xf12938580607 0.1 0 0 V2.3.1 ldp_bizproc - - 1 372 [[372][轮询处理错误:function(F973001),error_no(372),error_info([372][交易参数表:usps_exch_arg(唯一:idx_usps_exch_arg)记录不存在][p_exchange_type=3]),error_pathinfo(F973001->F1973001)记录不存在]]
```

平台事件日志(.eventlog)

记录重要事件，用于问题排查

事件日志信息(type=3)：		
通用信息(info)		
~~timestamp~~	~~日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS~~	~~必填~~
version	当前系统版本编号	必填
operator_id	操作员编号	选填，没有填"-"（减号）
charset	扩展信息中出入参的中文字符编码 0为GB18030，1为utf-8，不填为utf-8	选填，没有填"-"（减号，表示utf-8）
扩展信息(ext)		
ext	事件信息	必填

日志格式

```
log_version timestamp log_level processID-Ttid systemName-moduleName-MfunName-LlineNum formatType TP3 version operator_id charset ex
```

```
V2.0 2025-09-12 21:17:03.138490 EVENT 6779-T289 MInit-Lbizproc/bizproc_plugin_impl.cpp:116 FS TP3 V3.0 - 1 [实例 ldp_billy_33-xstA#0 初始化开始]
```

ext：事件号 事件信息

平台运维日志(.sysopslog)

操作系统、磁盘、内存、网络等基础设施和硬件错误； 运维参数设置错误

系统运维日志信息(type=6)：		
通用信息(info)		
trace_id	区分调用链消息的唯一标识，只能是字母和数字的组合； 一个请求的唯一标识，一个调用链中只有一个唯一的TraceID，所有TraceID相同的消息构成一个调用链	必填
span_id	跨度层级； 一个请求从发出到应答回来称为一个跨度（span），spanID用于区分不同的span，从0开始，每增加一个跨度，spanID就加一层，如0,0.1,0.1.1...，如果在一次消息处理中有多个请求被发出，则SpanID递增规则为：0.1,0.2,0.3...。 对于主动发起请求的服务填-1	必填
depot_id	**跨度里的层级；在对接BAR情况下，必须要填正确。在合并BAR和AR的场景下，用来标识请求和应答的depot_id，比如r0、a4等。隐含报文类型，r：请求，a：应答，即CRES的T2/F2插件的SR和CS可以合成请求，CR和SS可以合成应答**	**选填，没有填"-"（减号）**
~~timestamp~~	~~日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS~~	~~必填~~
response_status	**0表示成功，1表示失败**	必填
version	**当前系统版本编号**	必填
server_name	**服务名称（服务调用信息）：**	**必填**
	T2: function_id#sub_system_no#system_no#branch_no	
	T3: function_id#G#S#V#P	
	HTTP: url	
operator_id	操作员编号	选填，没有填"-"（减号）
cust_id	客户编号	选填，没有填"-"（减号）
charset	**扩展信息中出入参的中文字符编码**	**选填，没有填"-"（减号，表示utf-8）**
	0为GB18030，1为utf-8，不填为utf-8	
扩展信息(ext)		
error_no	**错误编号**	**必填**
error_info	**错误信息（包括相关参数）**	**必填**
error_infox	**记录错误扩展信息，一般为底层报错信息（错误号+错误信息）等**	**选填，没有填写""(或者不显示该字段)**

日志格式

```
log_version timestamp log_level processID-Ttid MfunName-LlineNum formatType TP6
trace_id spanid deport_id responseStatus version server_name operator_id cust_id
charset error_no error_info error_infox
```

```
参见程序运行日志。运维日志 traceid、span_id、depot_id可以为空。
```

平台警告日志(.warninglog)

证书快到期、队列排队、超时重试

警告日志信息(type=7)：		
通用信息(info)		
~~timestamp~~	~~日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS~~	~~必填~~
version	当前系统版本编号	必填
charset	扩展信息中出入参的中文字符编码	选填，没有填"-"（减号，表示utf-8）
	0为GB18030，1为utf-8，不填为utf-8	
扩展信息(ext)		
ext	告警信息	必填
error_no	错误编号	选填，没有填""（空）
error_info	错误信息	选填，没有填""（空）

日志格式

```
log_version timestamp log_level processID-Ttid systemName-moduleName-MfunName-
LlineNum formatType TP7 version 1 ext error_no error_infox
```

```
v2.0 2025-09-12 21:17:03.138490 WARN 68709-T7589 ldp_mproxy-auth-MauthFun-
L../utilities/ldp_auth/ldp_auth_impl.cpp:353 FS TP7 V3.0 1 4011 [找不到授权文件路径
./ldplicense/ 警告信息 没有那个文件或目录]
```

ext：事件号 事件信息，后面的error_no error_info为空（这里涉及到第三方解析问题，所以需要找长老再确认一下）

程序运行日志(.runtimelog)

包括请求执行中的程序缺陷错误和非服务技术组件（比如SDK）日志。前者用于开发人员定位和解决错误，后者主要用于排查异常。

程序运行日志信息(type=9)：		
通用信息(info)		
trace_id	区分调用链消息的唯一标识，只能是字母和数字的组合	必填
span_id	跨度层级	必填
depot_id	跨度里的层级；在对接BAR情况下，必须要填正确。在合并BAR和AR的场景下，用来标识请求和应答的depot_id，比如r0、a4等。隐含报文类型，r：请求，a：应答，即CRES的T2/F2插件的SR和CS可以合成请求，CR和SS可以合成应答	选填，没有填"-"（减号）
~~timestamp~~	~~日志产生的时间戳，格式：yyyy-MM-dd HH:mm:ss.SSSSSS~~	~~必填~~
response_status	0表示成功，1表示失败	必填
version	当前系统版本编号	必填
server_name	服务名称（服务调用信息）： T2: function_id#sub_system_no#system_no#branch_no T3: function_id#G#S#V#P HTTP: url	必填
operator_id	操作员编号	选填，没有填"-"（减号）
cust_id	客户编号	选填，没有填"-"（减号）
charset	扩展信息中出入参的中文字符编码 0为GB18030，1为utf-8，不填为utf-8	选填，没有填"-"（减号，表示utf-8）
扩展信息(ext)		
error_no	错误编号	必填
error_info	错误信息（包括相关参数）	必填
error_infox	记录错误扩展信息，一般为底层报错信息（错误号+错误信息）等	选填，没有填写""(或者不显示该字段)

日志格式

```
log_version timestamp log_level processID-Ttid systemName-moduleName-MfunName-LlineNum formatType TP9 trace_id spanid deport_id responseStatus version server_name operator_id cust_id charset error_no error_info error_infox
```

```
V2.0 2025-09-12 21:17:03.138490 ERROR 37859-T24 MForwardRequeset-Lfront_plugin_impl.cpp:1710 FS TP9 0xf12938580607 0.1 0 0 V3.0 ldp_front - - 1 1752 [转发请求失败（tyep=36，Function=501，8740)] [未知BackendId65535，消息功能号501类型36连接号1207959552]
```

error_infox中暂时不携带行号与错误码：8740 MSend-Lbackend_mgr.cpp:1710

原始示例：

```
8727 ERROR 191123 191327 消息路由BackendId未找到，是否Ust消息0 功能号 501 系统号 0 节点号 0(ldp_backend,Running,./backend/backend_mgr.cpp:3101,FindBackendId)
8740 WARN  191123 191327 未知BackendId 65535，消息 功能号 501 类型 36 连接号 1207959552(ldp_backend,Running,./backend/backend_mgr.cpp:2517,Send)
1542 ERROR 191123 191327 转发请求失败(type=36,FuncID=501,SysID=0,ConnID=1207959552,8740,ldp_front,Running,front_plugin_impl.cpp:1710,ForwardRequest)
```

附件：业务日志

[10.20.197.64_20250920-hsldp-ldp_sys_mdb_xn-usesbid62#0.bizlog]

方案设计

数据来源分析

log_version	固定值v2.0
time_stamp	上层调用日志接口的时间戳
log_level	日志宏传入。 链路日志固定INFO
server_info	支持外部自定义传入（后续可能需要支持）和 logproxy直接获取写入（进程号与线程号）
location	系统与功能名称由上层设置给CLogWriter，函数名、行号其使用宏，结合拼接位置。系统使用插件名、功能名称使用运行状态(待讨论)
format	logproxy内部确定，目前使用空格 方式，固定值FS
type	外部调用不同的日志宏传入不同的日志类型。格式为TP+编号
version	系统版本（插件版本？？）。
charset	logproxy内部配置，暂时固定值为1
traceid、span_id、deport_id、response_status	使用者传入logproxy, 存储于reader/writer/head中
server_name	插件名，上层传入
app_type	logproxy 内部使用数值9（ldp）
charset	logproxy 暂时内部写成固定值1（utf-8）
response_time	应答与请求的差值。（需要单独设计）暂停为-
拓展信息(ext)	error_no、error_info：上层的（服务模块）信息 error_infox：底层模块（remoter rcm）返回/回调的错误信息

模块功能实现

1. remoter提供回调接口，或者提供可以直接获取错误信息的接口，其他上层模块重载remoter的OnErrorEvent函数
2. logproxy实现新文件格式创建的功能，内部实现新格式的写入 。
3. 业务日志去掉/修改其他类型日志，只保存INFO、ERROR
4. 宏支持
5. 上层模块调用新日志宏

整体方案

方案一

需指定日志级别和日志类型，可以根据底层错误，决定自己是沿用以前的代码风格上层，根据错误号，调用不同的日志宏。

配置项可用方案二的设计，不过数据结构由每个插件维护。

兼容性考虑：需要产生新版本日志的地方调V2日志宏即可。

方案二

无需指定日志级别和日志类型

将配置信息（数据结构）导入CLogWriter，其根据不同的错误号描述，决定写入不同的文件

配置文件设计：

```
//配置于根节点
//支持模糊匹配
//支持配置校验
//如果错误号重叠，启动时告警，运行时优先作用于靠前的配置项
//闭区间
{
    "DebugNumRange":"1?-2?",
    "InfoNumRange":"40-100",
    "WarnNumRange":"100-1000",
    “RunTimeInfoNumRange”:"2000-4?44",
    “RunTimeErrorNumRange”:"5000-5???",
    “RunTimeFATALNumRange”:"6000-6???",
    "SysOpsErrorNumRange":"7000-7???",
    "SysOpsErrorNumRange":"8000-8???"
}
```

数据结构(AIGC协助)：

时间复杂度O(logn)

```
struct Range {
    int start;
    int end;
    std::string level;

    bool operator<(const Range& other) const {
        return start < other.start;
    }
};

class IntervalTree {
public:
    void addRange(const std::string& level, int start, int end) {
        ranges.insert({start, end, level});
    }

    std::string findLevel(int errorCode) {
        for (const auto& range : ranges) {
            if (errorCode >= range.start && errorCode <= range.end) {
                return range.level;
            }
        }
        return "Unknown";
    }

private:
    std::set<Range> ranges;
};
```

方案三（方案二的优化版）

不进行单独的文件配置，使用message_level中的值。

错误号字典新增字段表明日志类型，值参照：后续设计中的：LoggerType

默认值为0。使用原始的逻辑。

模块设计

front

1.重载remoter回调函数实现方法

错误描述示例：

```
)10751  3269 WARN  191123 191389 与服务端网络连接失败，服务端地址 192.168.86.102:31118，本端地址 192.168.86.102:48844，系统错误 111 拒绝连接(ConnIndex=1,initiator.cpp:1717,OnError)
```

回调行为需特殊处理。

格式化错误信息已经由（底层）remoter完成

```
int32_t OnErrorEvent(const ErrorInfoxContext& errorInfoxContext)
{
    //根据错误号结合当前状态决定要落入的日志类型。
    //告警日志
    m_lpILogProxy->WriteV2(nErrorNo，WARN, const wchar_t *lpErrorInfo);
    //
    可以直接调用
}
```

2. 打点实现

   V2版本的日志生成调用EndTimeStampV2接口，补充错误号与错误信息(如果外部可以传入)。

remoter

remoter中包含以下几种类型的错误。

实时监测、启动停止期、运行期（消息触发）。

实时监测

实时监测采用回调的方式。

```
class ILdpRemoterCallBack
{
    //错误信息回调
    virtual int32_t OnErrorEvent(const ErrorInfoxContext& errorInfoxContext)
noexcept = 0;
}
```

实时监测类，**remoter不用调用日志接口**，只需要上送即可。

启动停止

方案一：直接返回错误码与错误信息，函数调用层级比较深

方案二：try catch方案，外部都要捕获，外部改动点比较大

方案三：线程局部存储（使用方案三）

错误信息上下文

```
struct ErrorInfoxContext
{
    wchar* szLineInfo = nullptr;//为了后续拼接也是用wchar*
    int32_t errorNum = 0;
    wchar* szErrorInfo = nullptr;
}
```

提供外部获取接口

```
class ILdpRemoter
{   //获取当前线程最后一次错误信息
    virtual ErrorContext* GetLastErrorContext() noexcept = 0;
}
```

变量声明

```
class ILdpRemoter
{
private:
    //线程局部存储
    thread_local ErrorContext errorContext;
}
```

逻辑实现：

直接调用LOG_V2宏，通过logwriter、mgsformat获取位置、错误信息，并且设置到errorContext中。由外部获取

运行期

同上。

log_writer

日志错误信息存储与加载

```
class CMsgDef
{
public:
    uint32_t GetLogType(int32_t nErrorNo);
    int32_t LoadLevlAndLogType();
private:
    cosnt uint32_t         m_uDefaultLogType = 0;
    std::map<int32_t, int> m_MapErrType;//<<错误号，日志类型>>
}
```

为了支持运维日志、程序运行日志中包含链路信息，所以需要支持reader以及writer传入

```
通过错误号日志类型，决定调用后续的宏
//适用于debug warn event
#define LOG_V2（nErrorNo,...)
LDPLOGGER_UNLIKELY_V2(m_lpLogWriter,nErrorNo,LEVEL_INFO,##_VA_ARGS_)

//下面3个适用于runtime sysops
#define LOG_Reader_V2（nErrorNo,szErrorInfoEx,lpLdpReader...)
```

```
#define LOG_Writer_V2（nErrorNo,szErrorInfoEx,lpLdpWriter...)
//由于remoter可能没有解包器
#define LOG_Head_V2（nErrorNo,szErrorInfoEx,lpLdpHead...)

//调试、警告、事件不需要链路信息，那么不需要解包器
//.debuglog
#define LOG_DEBUG_V2（nErrorNo,...)
LDPLOGGER_UNLIKELY_V2(m_lpLogWriter,nErrorNo,LEVEL_DEBUG,##_VA_ARGS_)
//（.eventlog）
#define LOG_EVENT_V2（nErrorNo,...)
LDPLOGGER_UNLIKELY_V2(m_lpLogWriter,nErrorNo,LEVEL_EVENT,##_VA_ARGS_)
//（.warninglog）
#define LOG_WARN_V2（nErrorNo,...)
LDPLOGGER_UNLIKELY_V2(m_lpLogWriter,nErrorNo,LEVEL_WARN,##_VA_ARGS_)

由于运行日志和运维日志需要链路信息，那么需要传入解包器
//（.runtimelog）支持reader传入
#define LOG_INFO_RUN_Reader_V2（nErrorNo,szErrorInfoEx,lpLdpReader...)
LDPLOGGER_UNLIKELY(m_lpLogWriter,nErrorNo,szErrorInfoEx,lpLdpReader,LEVEL_INFO,#
#_VA_ARGS_)
#define LOG_ERROR_RUN_Reader_V2（nErrorNo,szErrorInfoEx,lpLdpReader...)
LDPLOGGER_UNLIKELY(m_lpLogWriter,nErrorNo,szErrorInfoEx,lpLdpReader,LEVEL_WARN,#
#_VA_ARGS_)
#define LOG_FATAL_RUN_Reader_V2（nErrorNo,szErrorInfoEx,lpLdpReader...)
LDPLOGGER_UNLIKELY(m_lpLogWriter,nErrorNo,szErrorInfoEx,lpLdpReader,LEVEL_FATAL,
##_VA_ARGS_)

//同理也要支持writer Head指针传入(实现省略)
LOG_INFO_RUN_??_V2（运行日志）（.runtimelog）
LOG_ERROR_RUN_??_V2（运行日志）（.runtimelog）
LOG_FATAL_RUN_??_V2（运行日志）（.runtimelog）

//参见运行日志（实现省略）
LOG_ERROR_SYS_??_V2（运维警告日志）(.sysopslog)
LOG_FATAL_SYS__??V2（运维严重错误日志）(.sysopslog)
```

内部函数实现(伪代码):

```
class CLogWriter{
public:

    cosnt wchar* GetLocationInfo()
    {
        return strLocation.c_str();
    }
    //这里仅描述Reader传入的方法，需补充writer和head类型的函数实现
    void CLogWrite::LogV2(ILdpMsgReader* lpLdpMsgReader, LoggerType logType,
int32_t nErrorNo, int32_t nLevel, wchar_t *szErrorInfox, const char *szLine,
cosnt char* szFunction,...)
    {
        //拼接位置信息
        FormatLocationInfo()
        //格式化错误信息拼接ErrorInfo和szErrorInfox
        FormatExtInfo()
        //调用logproxy接口
        m_lpILogProxy->WriteV2(lpLdpMsgReader, logType, nErrorNo, nLevel,
m_lpMesgFormat-       >GetMessage(),szLocation)
```

```
    }

    //szErrorInfox预留 业务暂时不用传入
    void CLogWrite::BizLogV2(ILdpMsgReader* lpLdpMsgReader, LoggerType logType,
int32_t nErrorNo, int32_t nLevel, wchar_t *szErrorInfox, const char *szLine,
cosnt char* szFunction,...)
    {
        //拼接位置信息
        FormatLocationInfo()
        //格式化错误信息拼接ErrorInfo和szErrorInfox
        FormatExtInfo()
        //调用logproxy接口
        m_lpILogProxy->WriteV2(lpLdpMsgReader, logType, nErrorNo, nLevel,
m_lpMesgFormat-       >GetMessage(),szLocation)
    }
    //提供设置系统和功能的接口用于拼接位置信息
    void SetSystemName(cont char* szSystemName)
    {

    }
    void SetModulemName(cont char* szModulemName)
    {

    }
privte:
    thread_local std::wstring strLocation;
}
```

业务日志已经支持reader，宏无需改动,见biz_data.h头文件。暂时不支持业务日志传入底层错误信息。

logproxy

接口

```
class ILogProxy{
public:
//除了业务日志、链路日志，都用这个接口
//较原始接口拆分原来错误信息为位置信息+错误描述(包含当前错误和底层错误)，消息操作对象
    virtual int32_t WriteV2(ILdpMsgReader *lpMsgReader,LoggerType logType,
int32_t nErrorNo, LogLevel eLevel, const const wchar_t *lpExt, char* lpLocation,
uint32 iFlag = OUTPUT_FILE) = 0;

    virtual int32_t WriteV2(ILdpMsgWriter *lpMsgWriter,LoggerType logType,
int32_t nErrorNo, LogLevel eLevel, const const wchar_t *lpExt, char* lpLocation,
uint32 iFlag = OUTPUT_FILE) = 0;
}

class ILogProxyEx{
public:
//业务日志接口
//较原始接口拆分原来错误信息为位置信息+错误描述(包含当前错误和底层错误)
    virtual int32_t WriteBizV2(ILdpMsgReader *lpMsgReader,int32_t nError,
LogLevel eLevel, const wchar_t *lpExt, const char* lpLocation)

//.tracelog 支持writer与reader传入，补充错误信息
    virtual int32_t EndTimeStampV2(ILdpMsgWriter* lpLdpMsgWriter, ITimeStamp*
lpTimeStamp,int nErrorNo, const wchar_t* lpErrorInfo)  = 0;
```

```
    virtual int32_t EndTimeStampV2(ILdpMsgReader* lpLdpMsgReader, ITimeStamp*
lpTimeStamp,int nErrorNo, const wchar_t* lpErrorInfo)  = 0;
}
```

新增日志类型

```
enum LoggerType :uint32t
{
    DEBUGLOG=0,//.debuglog
    TRACELOG=1,//.tracelog
    BIZLOG=2,//.bizlog
    EVENTLOG=3,//.eventlog
    SYSOPSLOG=6, //.sysopslog
    WARNINGLOG=7,//.warninglog
    RUNTIMELOG=9,//.runtimelog
}
```

```
enum LogLevel : uint32_t
{
    DEBUG=0x01,
    INFO=0x02,
    WARN=0x04,
    ERROR=0x08,
    FATAL=0x10,
}
```

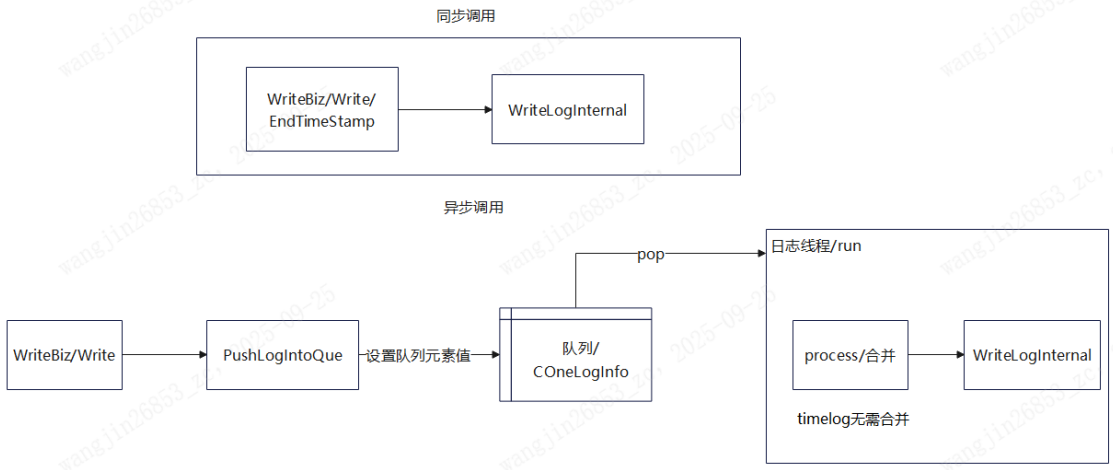
创建文件

```
//声明新的CLogWriter对象
class ClogProxyImpl
{
privte:
    CLogWrite *m_lpDebugLogWrite{nullptr};
    CLogwrite*m_lpTraceLogWrite{nullptr};
    CLogwrite*m_lpTimeLogwrite[nullptr];
    CLogWrite*m_lpBizLogWrite[nullptr];
    CLogWrite *m_lpEventLogWrite(nullptr};
    CLogwrite *m_lpSysOpsLogWrite{nullptr};
    CLogWrite *m_lpWarningLogWrite(nullptr};
    CLogwrite *m_lpRuntimeLogWrite{nullptr};
}

int32_t CLogProxyImpl::Init(const char *lpWorkDir, const char* *lpFileName)
{
    //todo创建新文件 参考log创建逻辑
    //.tracelog无需考虑合并。其他日志需要考虑合并
}
```

日志写入

线程模型




```
//所有信息log_version timestamp log_level processID-Ttid systemName-moduleName-
MfunName-LlineNum formatType TP1 trace_id spanid deport_id module_type
responseStatus version serverName appType charset error_no error_info error_infox
app_protocol req_body rep_body refs

//所以日志最终写入公用一个函数
int32_t WriteLogInternalV3(
struct timeval *tv,//时间戳信息，支持异步同步
LogLevel eLevel, //日志等级
int32_t tid,//线程号
const char* szlocation, //位置信息
LoggerType logType, //日志类型
const uint8_t *lpTraceId,//链路ID
uint32_t uDeportId,//
uint64_t *lpSpanId,//
uint32_t uSpandIdLevel, //错误号
const wchar_t* lpErrorInfo,//错误信息
size_t nSrcLen,//错误信息字符串长度
TimeStamp *lpTimeStamp,//timelog特有的
)
{
    //日志头的拼接逻辑一致，建议封装成一个函数
    //根据不同的日志类型拼接不同的字符串
    //需要考虑拓展头和可选头的traceid
}
```

兼容性设计

按照分期计划，logproxy需兼容原始日志输出和新版日志输出。

第三方兼容？？

补充

模块依赖说明

如果需要使用新版的日志格式，使用者需要依赖logproxy和hsldp

测试点设计

1. 兼容性测试，保证以前的日志正常输出
2. 查看新版本格式是否与描述一致
3. 参照错误列表描述 查看落入的文件是否正确
4. 链路日志需能在日志中心上正常采集绘制

非功能测试点设计

验证新版本链路日志对高性能下tps的影响

约束说明

只有调用了新版本日志宏，并且配置了错误号范围才能输出到V2日志中。

模块依赖说明

发布版本	存在依赖的模块	被依赖模块	被依赖模块发布版本	备注

工作量评估

任务序号	开发任务简述	开发工时评估（人日）
logproxy	支持新版日志	7
hsldp	提供日志宏	3
remoter	确定错误号范围，回调错误信息	3
front	对底层错误判断，调用新日志宏	2

遗留问题清单

评审记录

评审时间 20250922

评审人：董攀峰、胡细笔

评审纪要：

1. 有些格式没有统一，多余时延字段和业务信息还存在。（需要让同长老再确认下）
2. 确认链路原则，为什么非链路日志里面，也包含链路信息。链路信息与其他日志信息有重叠的地方，导致一个错误信息再很多地方被重复记录。（需要让同长老再确认下）
3. []的方式可能遇到转义的问题，内部格式方案，待格式确定后确认。
4. 进程名改成进程号

5. 暂停location格式：插件名-函数名-行号 （确认方案中接口跟方案的区别）
6. 后续需要设计补充的

   全部格式确定，每个参数怎么传，哪个模块来做，接口设计。

AP列表：

序号	AP项	责任人	计划完成时间